PaperQA : un assistant de questions-réponses sur des articles de recherche (RAG)

Lamyae Ez-Zaroura

Octobre 2026

Résumé

PaperQA est un projet personnel qui répond à des questions sur des PDF en citant le fichier et la page, et qui refuse de répondre quand l'information est absente. Ce rapport décrit l'architecture, le protocole d'évaluation (20 questions annotées sur un rapport de 21 pages) et les résultats : 16 réponses correctes sur 17, 17 citations sur 17 à la bonne page, 3 pièges sur 3 refusés, contre 1 réponse correcte sur 17 pour le même LLM sans documents. Les limites de l'évaluation sont discutées.

1 Objectif

Un LLM seul ne connaît pas les documents récents ou privés et peut inventer. L'objectif est de construire, de bout en bout et avec des outils gratuits, un système *RAG* (*Retrieval-Augmented Generation*) : retrouver les passages utiles, les fournir au LLM, et obtenir une réponse sourcée. Le projet vise aussi à *mesurer* la qualité du système plutôt qu'à l'affirmer.

2 Architecture

Indexation. Les pages sont extraites avec PyMuPDF (`pypdf` produisait des mots collés), découpées en morceaux de 1200 caractères avec 200 de chevauchement, transformées en vecteurs avec `all-MiniLM-L6-v2` (`sentence-transformers`) et stockées dans une base Chroma persistante. Chaque morceau garde son fichier source et son numéro de page.

Question. La question est vectorisée ; les $k = 8$ morceaux les plus proches sont insérés dans un prompt qui impose de n'utiliser que ces extraits, de citer `[fichier p.N]` et de répondre « Je ne trouve pas la réponse dans les documents. » sinon. Le LLM est Gemini (API gratuite, modèle `gemini-3.5-flash-lite` pour l'évaluation), appelé avec délai maximum et réessais en cas d'erreur 429/503.

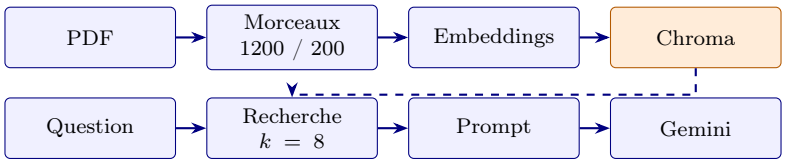


FIGURE 1 – Ligne du haut : indexation (une fois). Ligne du bas : traitement d'une question.

Deux interfaces. Une API FastAPI expose `POST /ask` (validation Pydantic, erreurs 422/503) et `GET /health` : elle s'adresse à des programmes (page web, script). Un serveur MCP (SDK Python `mcp`, transport standard entrée/sortie) expose la recherche seule, via les outils `search_papers` et `list_documents` : un assistant IA décide lui-même de l'appeler puis rédige la réponse à partir des passages. Le serveur n'appelle pas Gemini. Tout affichage parasite est redirigé vers `stderr`, car un `print` sur la sortie standard casserait le protocole.

Tests. 16 tests `pytest` s'exécutent sans réseau (LLM et base simulés) : ingestion, RAG, API et serveur MCP.

3 Protocole d'évaluation

Corpus et questions. Un rapport de 21 pages sur l'évolution de la violence en Afrique du Nord et de l'Ouest. 20 questions écrites à la main : 8 factuelles, 5 de raisonnement (plusieurs passages), 4 en français sur un texte anglais, 3 pièges (réponse absente ou hors sujet). Chaque question a une réponse attendue, la ou les pages correspondantes et des mots-clés (listes de variantes) qui doivent apparaître dans la réponse.

Mesures. *hit@k (page)* : une partie de la bonne page figure parmi les k premiers résultats. *hit@k (contenu)* : le morceau retrouvé contient lui-même les chiffres attendus, ce qui est plus exigeant. Pour les réponses : correctes, refus à tort, réponses fausses, inventions sur les pièges, citation de la bonne page, et comparaison avec le même LLM sans documents.

4 Résultats

4.1 Taille des morceaux

Découpage (taille / chevauchement)	hit@3	hit@5	hit@8
350 / 70	10/17	11/17	12/17
500 / 100	6/17	11/17	11/17
800 / 150	10/17	12/17	15/17
1200 / 200 (retenu)	**12/17**	**14/17**	**16/17**

Table 1 – Recherche seule, hit@k (contenu), 17 questions avec réponse.

À 800/150, la question sur le nombre de décès par événement était refusée à tort : la bonne page était retrouvée, mais le morceau contenant la phrase utile, mélangé à un autre paragraphe, était classé trop bas. Le hit@k par page masquait ce problème, d'où l'ajout de la mesure par contenu. Cette mesure avantage un peu les grands morceaux, car elle exige tous les chiffres dans un même morceau.

4.2 Évaluation complète

La bonne page est retrouvée dans 17 cas sur 17 dès $k = 5$; le morceau contenant la réponse l'est dans 16 cas sur 17 à $k = 8$.

Lecture du « sans RAG ». Le rapport date de 2026 : le LLM seul ne peut pas connaître ses chiffres. Le résultat de 1/17 illustre l'intérêt du RAG pour des documents récents ou privés, pas une supériorité générale du RAG. Le seul succès sans RAG est une question d'explication qui n'exige pas de chiffre.

Mesure	Avec RAG	Sans RAG
Réponses correctes (17 questions)	**16/17 (94 %)**	1/17 (6 %)
Refus à tort	0/17	–
Réponses fausses	1/17	–
Citation de la bonne page	17/17	–
Pièges bien refusés	3/3, 0 invention	–

TABLE 2 – $k = 8$, `gemini-3.5-flash-lite`. Par type : factuelles 8/8, français 4/4, raisonnement 4/5.

Erreur restante. Pour la question sur les projections 2026 du Mali et du Burkina Faso, le système répond « plus de 40 % » et « baisse de 30 % », fidèle au texte retrouvé, alors que le rapport donne 42 % et 27 %, probablement dans un tableau absent des 8 morceaux retrouvés. C'est une limite réelle de la recherche sur les tableaux.

Corrections de la notation. Au premier passage complet, 14/17 réponses étaient jugées correctes. Deux des trois erreurs venaient de la notation : un mot-clé sans accent (« Nigeria » pour « Nigéria »), corrigé en ignorant les accents, et un chiffre exigé pour une question d'explication, remplacé par des mots-clés d'explication. Les réponses n'ont pas changé ; le score passe à 16/17. La troisième erreur a été conservée.

5 Difficultés rencontrées

— Un refus à tort venait de la recherche et non du LLM (bon passage classé 6e pour 4 transmis) : k porté à 8 et outil `inspect_question` pour inspecter les résultats.
— Extraction PDF : passage de `pypdf` à PyMuPDF.
— Quota gratuit : erreurs 429/503, réponses mises en cache, délai maximum, option `-pause` et changement de modèle.

6 Limites et perspectives

L'évaluation porte sur un seul document, un seul modèle et 17 questions que j'ai écrites moi-même : les pourcentages sont des ordres de grandeur. La notation par mots-clés est imparfaite. Pas d'OCR pour les PDF scannés ; tableaux et figures lus comme du texte brut ; embeddings orientés anglais. Pistes : frontend (Streamlit), embeddings multilingues (comparaison mesurée), reranking, traitement des tableaux, Docker et intégration continue.